Paperclip Swarm - □□□□□□

1. □□□□
   1.1 □□
   1.2 □□□□
   1.3 □□□
2. □□□□
   2.1 □□□□□
   2.2 □□□□
3. □□□□□□□□
   3.1 pswarm-core - □□□□□□
   3.2 pswarm-runtime - Agent □□□
   3.3 pswarm-scheduler - □□□□□
   3.4 pswarm-network - QUIC □□□
   3.5 pswarm-api - HTTP/gRPC API
4. □□□□
   4.1 □□□□
   4.2 □□□□
   4.3 □□□□
5. □□□□
   5.1 □□□□
   5.2 Kubernetes □□
6. □□□□□□
   Phase 1: □□□□□2□□
   Phase 2: □□□□□2□□
   Phase 3: API □□□□2□□
   Phase 4: □□□□□□□□2□□
7. □□
   7.1 □□□
   7.2 □□□□

Paperclip Swarm - □□□□□□

	v0.1.0-alpha
	2026-03-11
	Rust (Edition 2021)
□□ Rust □□	1.75.0

---

1. □□□□

1.1 □□

□□□□□□□□□□□□□□□□□□ Agent Swarm □□□□□□□□□□□□□□□

1. □Agent □□□□□□□□□□□□□□□
2. □□□□□ + Mesh □□□□

3. □□□□□□□□□□□□□□□
4. □□□□□□ Kubernetes □□

1.2 □□□□

- □□□ **Agent** □YAML/JSON □□□□
- **QUIC** □□0-RTT□□□ mTLS□□□□□
- **WASM** □□□□□□□□□□□□□□
- □□□□□□□□□□□□□□□□□□□□
- □OpenTelemetry□Prometheus□□□□□□

1.3 □□□

	tokio	^1.35
HTTP □□	axum	^0.7
QUIC □□	quinn	^0.10
	prost + serde	-
	sqlx (PostgreSQL)	^0.7
	redis	^0.24
WASM □□□	wasmtime	^17.0
	config + serde_yaml	-
	tracing + tracing-subscriber	-
	metrics + metrics-prometheus	-

---

2. □□□□

2.1 □□□□□

```
┌──────────────
```

```
┌──────────────
```

```
│   │ │   │
│  │  │ │Instance│ │ │ │Instance│ │ │ │Instance│ │ │   │  Instance
│   │ │   │
│  │  │ └────────┘ │ │ └────────┘ │ │ └────────┘ │ │
└──────────────┘   │ │   │
│  │  └──────────────┘ └──────────────┘ └──────────────┘
└────────────────────┘ │   │
│
└──────────────────────────────────────────────────────────────────
   │
│
│
│
┌──────────────────────────────────────────────────────────────────
   │
│  │                     Mesh Network (QUIC)
   │
│  │         ┌──────────────┐  Gossip Protocol  ┌──────────────┐
   │
│  │         │   Agent A    │◄──────────────────►│   Agent B    │
   │
│  │         └──────────────┘                   └──────────────┘
   │
│  │                  ▲                              ▲
   │
│  │                  └──────────────┬───────────────┘
   │
│  │                                 │
   │
│  │                          ┌──────────────┐
   │
│  │                          │   Agent C    │
   │
│  │                          └──────────────┘
   │
│
└──────────────────────────────────────────────────────────────────
   │
└──────────────────────────────────────────────────────────────────

         │
         ▼
┌──────────────────────────────────────────────────────────────────

│                       Storage Layer
│
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐
┌──────────────┐     │
│  │    Redis     │  │  PostgreSQL  │  │    MinIO     │  │   Etcd/ZK
│     │
│  │ (□□□□/□□) │  │  (□□□□□□) │  │ (□□□□)    │  │ (□□□□□□)  │     │
│  └──────────────┘  └──────────────┘  └──────────────┘
└──────────────┘     │
└──────────────────────────────────────────────────────────────────
```

2.2 □□□□

```
paperclip-swarm/
├── crates/
```

```
│   ├── pswarm-core/          # □□□□□ trait
│   ├── pswarm-runtime/       # Agent □□□ (WASM/Container)
│   ├── pswarm-scheduler/     # □□□□□
│   ├── pswarm-network/       # QUIC □□□
│   ├── pswarm-discovery/     # □□□□
│   ├── pswarm-storage/       # □□□□
│   ├── pswarm-api/           # HTTP/gRPC API
│   ├── pswarm-cli/           # □□□□□
│   └── pswarm-operator/      # K8s Operator
├── proto/                    # Protocol Buffer □□
├── docs/                     # □□
├── examples/                 # □□□□
└── deployments/              # □□□□
```

---

3. □□□□□□□□

3.1 pswarm-core - □□□□□□

: crates/pswarm-core/src/lib.rs

3.1.1 Agent □□□□

```rust
//! □□□□□□
//!
//! □□□□□□□□□□□□□□□□□□ trait

use std::collections::HashMap;
use std::time::{Duration, SystemTime};
use serde::{Deserialize, Serialize};
use uuid::Uuid;

/// □□□□□□□□□□□□
pub type AgentId = Uuid;
pub type TaskId = Uuid;
pub type SessionId = Uuid;
pub type NodeId = Uuid;

/// Agent □□□□
#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
pub enum AgentState {
    /// □□□□
    Creating,
    /// □□□□□□□□
    Idle,
    /// □□□
    Running,
    /// □□
    Paused,
    /// □□□□
    Terminating,
    /// □□□
    Terminated,
    /// □□
    Failed,
}

/// Agent □□□□□
```

```rust
#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
pub enum RuntimeType {
    /// WASM □□□□□□□
    Wasm,
    /// □□□□□
    Container,
    /// □□□□□□□□□□□□□
    Process,
}

/// Agent □□□□
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct ResourceQuota {
    /// CPU □□ (millicores)
    pub cpu_limit: u32,
    /// □□□□ (bytes)
    pub memory_limit: u64,
    /// □□□□□□ (bytes)
    pub ephemeral_storage: u64,
    /// GPU □□□□□□
    pub gpu_limit: Option<u32>,
    /// □□□□□□□□
    pub max_concurrent_tasks: usize,
    /// □□□□□□
    pub idle_timeout: Duration,
    /// □□□□□□□□□□□□
    pub max_lifetime: Duration,
}

impl Default for ResourceQuota {
    fn default() -> Self {
        Self {
            cpu_limit: 1000,  // 1 core
            memory_limit: 512 * 1024 * 1024,  // 512MB
            ephemeral_storage: 1 * 1024 * 1024 * 1024,  // 1GB
            gpu_limit: None,
            max_concurrent_tasks: 1,
            idle_timeout: Duration::from_secs(300),  // 5□□
            max_lifetime: Duration::from_secs(3600),  // 1□□
        }
    }
}

/// Agent □□□□
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Capability {
    /// □□□□
    pub name: String,
    /// □□□□
    pub description: String,
    /// □□□□□JSON Schema□
    pub input_schema: serde_json::Value,
    /// □□□□□JSON Schema□
    pub output_schema: serde_json::Value,
    /// □□□□
    pub timeout: Duration,
    /// □□□□□□□□
    pub require_human_approval: bool,
}
```

```rust
/// Agent □□□□□□□□□
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct AgentSpec {
    /// API □□
    pub api_version: String,
    /// □□□
    pub metadata: AgentMetadata,
    /// □□□□□
    pub runtime: RuntimeConfig,
    /// □□□□
    pub resources: ResourceQuota,
    /// □□□□
    pub capabilities: Vec<Capability>,
    /// □□□□
    pub handoff_rules: Vec<HandoffRule>,
    /// □□□□
    pub model: Option<ModelConfig>,
    /// □□□□
    pub env: HashMap<String, String>,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct AgentMetadata {
    /// □□□□□□
    pub name: String,
    /// □□□□
    pub namespace: String,
    /// □□
    pub version: String,
    /// □□
    pub labels: HashMap<String, String>,
    /// □□
    pub annotations: HashMap<String, String>,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct RuntimeConfig {
    /// □□□□□
    pub runtime_type: RuntimeType,
    /// WASM □□□□□□□
    pub image: String,
    /// □□□□□□□□□□□□□
    pub entrypoint: Option<Vec<String>>,
    /// □□□□□□
    pub args: Option<Vec<String>>,
    /// □□□□
    pub working_dir: Option<String>,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct ModelConfig {
    /// □□□□openai, anthropic, local □□
    pub provider: String,
    /// □□□□
    pub name: String,
    /// □□□□
    pub temperature: Option<f32>,
    /// □□ token
    pub max_tokens: Option<u32>,
    /// API □□□□□□ secret □□□
```

```rust
    pub api_key_secret: Option<String>,
}

/// □□□□
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct HandoffRule {
    /// □□□□□□CEL □□□
    pub condition: String,
    /// □□ Agent □□
    pub target_agent: String,
    /// □□□□□□□
    pub context_transfer: ContextTransferMode,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
pub enum ContextTransferMode {
    /// □□□□□□□
    Full,
    /// □□□□□
    Summary,
    /// □□□□□
    Filtered(Vec<String>),
}

/// Agent □□□□
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct AgentInstance {
    /// □□ ID
    pub id: AgentId,
    /// □□□□
    pub node_id: NodeId,
    /// Agent □□□□
    pub spec_name: String,
    /// □□□□
    pub state: AgentState,
    /// □□□□
    pub resource_usage: ResourceUsage,
    /// □□□□
    pub started_at: SystemTime,
    /// □□□□□□
    pub last_active_at: SystemTime,
    /// □□□□□□□ Mesh □□□
    pub mesh_addr: Option<String>,
    /// □□□□
    pub sessions: Vec<SessionId>,
}

#[derive(Debug, Clone, Default, Serialize, Deserialize)]
pub struct ResourceUsage {
    /// CPU □□□ (millicores)
    pub cpu_millicores: u32,
    /// □□□□□ (bytes)
    pub memory_bytes: u64,
    /// □□□□□ (bytes)
    pub storage_bytes: u64,
}

/// □□□□
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Task {
```

```rust
    /// □□ ID
    pub id: TaskId,
    /// □□□ ID□□□□□□□□
    pub parent_id: Option<TaskId>,
    /// □□□□
    pub task_type: String,
    /// □□□□
    pub input: serde_json::Value,
    /// □□□
    pub priority: TaskPriority,
    /// □□□□
    pub timeout: Duration,
    /// □□□ Agent □□
    pub required_capabilities: Vec<String>,
    /// □□□□□□□□□□□□□□
    pub labels: HashMap<String, String>,
    /// □□□□
    pub created_at: SystemTime,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
pub enum TaskPriority {
    Critical = 0,
    High = 1,
    Normal = 2,
    Low = 3,
}

/// □□□□
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct TaskStatus {
    /// □□ ID
    pub task_id: TaskId,
    /// □□□□
    pub state: TaskState,
    /// □□□ Agent
    pub assigned_agent: Option<AgentId>,
    /// □□□□
    pub started_at: Option<SystemTime>,
    /// □□□□
    pub completed_at: Option<SystemTime>,
    /// □□□□
    pub result: Option<serde_json::Value>,
    /// □□□□
    pub error: Option<String>,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
pub enum TaskState {
    Pending,
    Scheduled,
    Running,
    Paused,
    Completed,
    Failed,
    Cancelled,
    Timeout,
}

/// □□□□□□□□□□□□
```

```rust
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Session {
    /// □□ ID
    pub id: SessionId,
    /// □□□ ID
    pub root_task_id: TaskId,
    /// □□□ Agent □□
    pub agents: Vec<AgentId>,
    /// □□□□□ Agent
    pub current_agent: Option<AgentId>,
    /// □□□□
    pub messages: Vec<Message>,
    /// □□□□□
    pub context_variables: HashMap<String, serde_json::Value>,
    /// □□□□
    pub created_at: SystemTime,
    /// □□□□□□
    pub updated_at: SystemTime,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Message {
    /// □□ ID
    pub id: Uuid,
    /// □□□□Agent ID □ "user"□
    pub sender: String,
    /// □□□
    pub recipient: Option<String>,
    /// □□□□
    pub message_type: MessageType,
    /// □□
    pub content: serde_json::Value,
    /// □□□
    pub timestamp: SystemTime,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
pub enum MessageType {
    UserQuery,
    AgentResponse,
    ToolCall,
    ToolResult,
    HandoffRequest,
    HandoffAccept,
    SystemEvent,
}

/// □□ trait□Agent □□□□□□
#[async_trait::async_trait]
pub trait AgentLifecycle: Send + Sync {
    /// □□ Agent □□
    async fn create(&self, spec: AgentSpec) -> Result<AgentInstance,
AgentError>;

    /// □□ Agent
    async fn start(&self, agent_id: AgentId) -> Result<(),
AgentError>;

    /// □□ Agent
    async fn pause(&self, agent_id: AgentId) -> Result<(),
```

```
AgentError>;

            /// □□ Agent
            async fn resume(&self, agent_id: AgentId) -> Result<(),
AgentError>;

            /// □□ Agent
            async fn terminate(&self, agent_id: AgentId) -> Result<(),
AgentError>;

            /// □□ Agent □□
            async fn get_status(&self, agent_id: AgentId) ->
Result<AgentInstance, AgentError>;

            /// □□□□ Agent
            async fn list_agents(&self, filter: AgentFilter) ->
Result<Vec<AgentInstance>, AgentError>;
        }

        /// Agent □□□□
        #[derive(Debug, Clone, Default)]
        pub struct AgentFilter {
            pub namespace: Option<String>,
            pub spec_name: Option<String>,
            pub state: Option<AgentState>,
            pub node_id: Option<NodeId>,
            pub labels: HashMap<String, String>,
        }

        /// Agent □□□□
        #[derive(Debug, thiserror::Error)]
        pub enum AgentError {
            #[error("Agent not found: {0}")]
            NotFound(AgentId),

            #[error("Resource exhausted: {0}")]
            ResourceExhausted(String),

            #[error("Invalid spec: {0}")]
            InvalidSpec(String),

            #[error("Runtime error: {0}")]
            RuntimeError(String),

            #[error("Communication error: {0}")]
            CommunicationError(String),

            #[error("Timeout")]
            Timeout,

            #[error(transparent)]
            Other(#[from] anyhow::Error),
        }

        /// □□ trait□□□□□
        #[async_trait::async_trait]
        pub trait TaskScheduler: Send + Sync {
            /// □□□□
            async fn submit(&self, task: Task) -> Result<TaskId,
SchedulerError>;
```

```rust
            /// □□□□
            async fn cancel(&self, task_id: TaskId) -> Result<(),
SchedulerError>;

            /// □□□□□□□
            async fn get_status(&self, task_id: TaskId) ->
Result<TaskStatus, SchedulerError>;

            /// □□□□□□□
            async fn wait_for_completion(&self, task_id: TaskId) ->
Result<TaskStatus, SchedulerError>;
        }

        #[derive(Debug, thiserror::Error)]
        pub enum SchedulerError {
            #[error("Task not found: {0}")]
            NotFound(TaskId),

            #[error("No available agent for task")]
            NoAvailableAgent,

            #[error("Queue full")]
            QueueFull,

            #[error(transparent)]
            Other(#[from] anyhow::Error),
        }

        /// □□ trait□□□□□
        #[async_trait::async_trait]
        pub trait NetworkTransport: Send + Sync {
            /// □□□□□□□□ Agent
            async fn send_to(&self, agent_id: AgentId, message: Message) ->
Result<(), NetworkError>;

            /// □□□□□□□□ Agent
            async fn broadcast(&self, message: Message) -> Result<(),
NetworkError>;

            /// □□□□
            async fn receive(&self) -> Result<(AgentId, Message),
NetworkError>;
        }

        #[derive(Debug, thiserror::Error)]
        pub enum NetworkError {
            #[error("Connection refused")]
            ConnectionRefused,

            #[error("Agent unreachable: {0}")]
            Unreachable(AgentId),

            #[error("Timeout")]
            Timeout,

            #[error(transparent)]
            Other(#[from] anyhow::Error),
        }
```

3.2 pswarm-runtime - Agent □□□

: crates/pswarm-runtime/src/lib.rs

3.2.1 □□□□□□

```rust
//! Agent □□□□□
//!
//! □□□□ WASM/Container/Process □□□□□□□□□□□

use std::collections::HashMap;
use std::sync::Arc;
use tokio::sync::{Mutex, RwLock};
use wasmtime::{Engine, Module, Store};

use pswarm_core::*;

/// □□□□□□
pub struct RuntimeManager {
    /// WASM □□
    wasm_engine: Engine,
    /// □□□□
    module_cache: Arc<RwLock<HashMap<String, Module>>>,
    /// □□□□
    instances: Arc<Mutex<HashMap<AgentId, Box<dyn AgentRuntime>>>>,
    /// □□□
    resource_pool: Arc<ResourcePool>,
    /// □□
    config: RuntimeConfig,
}

#[derive(Debug, Clone)]
pub struct RuntimeConfig {
    /// WASM □□□□□□
    pub wasm_cache_dir: std::path::PathBuf,
    /// □□□□□□□□
    pub max_instances: usize,
    /// □□□□□
    pub warmup_instances: usize,
    /// □□□□□□containerd/cri-o□
    pub container_runtime: String,
    /// □□□□
    pub network_mode: NetworkMode,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum NetworkMode {
    /// □□□□
    Isolated,
    /// □□□□
    Shared,
    /// □□□□□□□□□□
    Host,
}

/// □□□ trait
#[async_trait::async_trait]
pub trait AgentRuntime: Send + Sync {
    /// □□ Agent ID
    fn agent_id(&self) -> AgentId;
```

```rust
            /// □□□□□□□□
            fn runtime_type(&self) -> RuntimeType;

            /// □□□□□□□
            async fn execute(&self, task: Task) -> Result<TaskResult,
RuntimeError>;

            /// □□□□□□□
            async fn start_session(&self, session: Session) -> Result<(),
RuntimeError>;

            /// □□□□□□□
            async fn send_message(&self, message: Message) -> Result<(),
RuntimeError>;

            /// □□□□□□□
            async fn receive_message(&self) -> Result<Message,
RuntimeError>;

            /// □□□□□□□
            fn resource_usage(&self) -> ResourceUsage;

            /// □□□□□
            async fn checkpoint(&self) -> Result<Checkpoint, RuntimeError>;

            /// □□□□□□□
            async fn restore(&self, checkpoint: Checkpoint) -> Result<(),
RuntimeError>;

            /// □□□□
            async fn graceful_shutdown(&self) -> Result<(), RuntimeError>;

            /// □□□□
            async fn force_terminate(&self) -> Result<(), RuntimeError>;
        }

        /// □□□□□□
        #[derive(Debug, Clone, Serialize, Deserialize)]
        pub struct TaskResult {
            /// □□ ID
            pub task_id: TaskId,
            /// □□□□
            pub status: TaskStatus,
            /// □□□□
            pub output: Option<serde_json::Value>,
            /// □□□□
            pub metrics: ExecutionMetrics,
        }

        #[derive(Debug, Clone, Default, Serialize, Deserialize)]
        pub struct ExecutionMetrics {
            /// □□□□ (ms)
            pub startup_latency_ms: u64,
            /// □□□□ (ms)
            pub execution_time_ms: u64,
            /// □□□□ (bytes)
            pub memory_peak_bytes: u64,
            /// Token □□□□LLM □□□
            pub token_usage: Option<TokenUsage>,
```

```rust
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct TokenUsage {
    pub prompt_tokens: u32,
    pub completion_tokens: u32,
    pub total_tokens: u32,
}

/// □□□□□□
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Checkpoint {
    /// □□□ ID
    pub id: Uuid,
    /// Agent ID
    pub agent_id: AgentId,
    /// □□□□
    pub created_at: SystemTime,
    /// □□□□
    pub state_data: Vec<u8>,
    /// □□□□□WASM □□□□□
    pub memory_snapshot: Option<Vec<u8>>,
}

/// WASM □□□□□
pub struct WasmRuntime {
    agent_id: AgentId,
    store: Arc<Mutex<Store<WasmContext>>>,
    instance: wasmtime::Instance,
    resource_usage: Arc<RwLock<ResourceUsage>>,
    metrics_collector: MetricsCollector,
}

struct WasmContext {
    /// □□□□
    stdout: Vec<u8>,
    /// □□□□
    stderr: Vec<u8>,
    /// □□□□□□
    host_functions: HostFunctions,
}

/// □□□□□WASM □□□□□□□□□□□
pub struct HostFunctions {
    /// HTTP □□
    pub http_client: Arc<dyn HttpClient>,
    /// □□
    pub logger: Arc<dyn Logger>,
    /// KV □□
    pub kv_store: Arc<dyn KVStore>,
    /// LLM □□
    pub llm_client: Arc<dyn LLMClient>,
}

/// □□□□□□□□
pub struct ContainerRuntime {
    agent_id: AgentId,
    container_id: String,
    /// containerd/cri-o □□□
    client: Arc<dyn ContainerClient>,
```

```rust
        /// □□□□
        network_endpoint: String,
        resource_usage: Arc<RwLock<ResourceUsage>>,
    }

    /// □□□□□□□□□□
    pub struct ResourcePool {
        /// □ spec □□□□□□□
        warm_instances: Arc<RwLock<HashMap<String, Vec<Box<dyn
AgentRuntime>>>>>,
        /// □□□□□
        max_warm_per_spec: usize,
        /// □□□□□
        resource_manager: Arc<ResourceManager>,
    }

    impl ResourcePool {
        /// □□□□□□□□
        pub async fn acquire(&self, spec: &AgentSpec) -> Result<Box<dyn
AgentRuntime>, PoolError> {
            // 1. □□□□□□□□□
            if let Some(instance) = self.try_get_warm(spec).await {
                // □□□□
                instance.reconfigure(spec).await?;
                return Ok(instance);
            }

            // 2. □□□□□
            self.create_new(spec).await
        }

        /// □□□□□□□□
        pub async fn release(&self, runtime: Box<dyn AgentRuntime>) {
            let spec_name = runtime.spec_name();
            let mut instances = self.warm_instances.write().await;

            let pool = instances.entry(spec_name).or_default();
            if pool.len() < self.max_warm_per_spec {
                pool.push(runtime);
            } else {
                // □□□□□□□□
                drop(runtime);
            }
        }
    }

    /// □□□□□
    pub struct ResourceManager {
        /// □□□□□
        total_quota: ResourceQuota,
        /// □□□□□
        allocated: Arc<RwLock<ResourceUsage>>,
        /// □□□□□K8s □□□
        node_info: Option<NodeInfo>,
    }

    impl ResourceManager {
        /// □□□□□□□□□□
        pub async fn check_availability(&self, required: &ResourceQuota)
-> Result<(), ResourceError> {
```

```
                let allocated = self.allocated.read().await;
                let total = &self.total_quota;

                if allocated.cpu_millicores + required.cpu_limit >
total.cpu_limit {
                    return Err(ResourceError::CpuExceeded);
                }

                if allocated.memory_bytes + required.memory_limit >
total.memory_limit {
                    return Err(ResourceError::MemoryExceeded);
                }

                Ok(())
            }

            /// □□□□
            pub async fn allocate(&self, quota: &ResourceQuota) ->
Result<ResourceHandle, ResourceError> {
                self.check_availability(quota).await?;

                let mut allocated = self.allocated.write().await;
                allocated.cpu_millicores += quota.cpu_limit;
                allocated.memory_bytes += quota.memory_limit;

                Ok(ResourceHandle {
                    quota: quota.clone(),
                    allocator: Arc::downgrade(&self.allocated),
                })
            }
        }

        /// □□□□□□□□□□□
        pub struct ResourceHandle {
            quota: ResourceQuota,
            allocator: std::sync::Weak<RwLock<ResourceUsage>>,
        }

        impl Drop for ResourceHandle {
            fn drop(&mut self) {
                if let Some(allocator) = self.allocator.upgrade() {
                    // □□□□□□
                    let quota = self.quota.clone();
                    tokio::spawn(async move {
                        let mut allocated = allocator.write().await;
                        allocated.cpu_millicores -= quota.cpu_limit;
                        allocated.memory_bytes -= quota.memory_limit;
                    });
                }
            }
        }
```

3.3 pswarm-scheduler - □□□□□

: crates/pswarm-scheduler/src/lib.rs

```
        //! □□□□□
        //!
        //! □□□□□□□□□□□□□□□□□□□
```

```rust
use std::collections::{BinaryHeap, HashMap, VecDeque};
use std::sync::Arc;
use tokio::sync::{Mutex, Notify, RwLock};
use tokio::time::{interval, Duration, Instant};

use pswarm_core::*;

/// □□□□□
#[derive(Debug, Clone)]
pub struct SchedulerConfig {
    /// □□□□
    pub strategy: SchedulingStrategy,
    /// □□□□□□
    pub queue_capacity: usize,
    /// □□□□
    pub max_retries: u32,
    /// □□□□
    pub retry_backoff: Duration,
    /// □□□□□□□□□
    pub timeout_check_interval: Duration,
    /// □□□□□□□□□□
    pub enable_affinity: bool,
    /// □□□□□□□□□□□□□□□□
    pub enable_replication: bool,
}

impl Default for SchedulerConfig {
    fn default() -> Self {
        Self {
            strategy: SchedulingStrategy::Adaptive,
            queue_capacity: 10000,
            max_retries: 3,
            retry_backoff: Duration::from_secs(5),
            timeout_check_interval: Duration::from_secs(10),
            enable_affinity: true,
            enable_replication: false,
        }
    }
}

/// □□□□
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum SchedulingStrategy {
    /// □□
    RoundRobin,
    /// □□□□
    LeastConnections,
    /// □□□□
    ResourceWeighted,
    /// □□□□□□□
    Adaptive,
}

/// □□□□□
pub struct Scheduler {
    /// □□
    config: SchedulerConfig,
    /// □□□□□□□□□□□
    task_queue: Arc<Mutex<BinaryHeap<QueuedTask>>>,
```

```rust
        /// □□□□□
        running_tasks: Arc<RwLock<HashMap<TaskId, TaskHandle>>>,
        /// Agent □□□
        agent_manager: Arc<dyn AgentManager>,
        /// □□□□
        notify: Arc<Notify>,
        /// □□
        metrics: Arc<SchedulerMetrics>,
    }

    /// □□□□□□□□□□□□□□
    #[derive(Debug, Clone)]
    struct QueuedTask {
        /// □□□□□□□□□□
        priority: u8,
        /// □□□□
        enqueued_at: Instant,
        /// □□
        task: Task,
        /// □□□□
        retries: u32,
    }

    impl Ord for QueuedTask {
        fn cmp(&self, other: &Self) -> std::cmp::Ordering {
            // □□□□□□□□□ FIFO
            self.priority
                .cmp(&other.priority)
                .then_with(|| other.enqueued_at.cmp(&self.enqueued_at))
        }
    }

    impl PartialOrd for QueuedTask {
        fn partial_cmp(&self, other: &Self) ->
Option<std::cmp::Ordering> {
            Some(self.cmp(other))
        }
    }

    impl PartialEq for QueuedTask {
        fn eq(&self, other: &Self) -> bool {
            self.priority == other.priority && self.enqueued_at ==
other.enqueued_at
        }
    }

    impl Eq for QueuedTask {}

    impl Scheduler {
        pub fn new(config: SchedulerConfig, agent_manager: Arc<dyn
AgentManager>) -> Self {
            Self {
                config,
                task_queue: Arc::new(Mutex::new(BinaryHeap::new())),
                running_tasks: Arc::new(RwLock::new(HashMap::new())),
                agent_manager,
                notify: Arc::new(Notify::new()),
                metrics: Arc::new(SchedulerMetrics::default()),
            }
        }
```

```rust
            /// □□□□□□
            pub async fn start(&self) {
                let config = self.config.clone();
                let queue = self.task_queue.clone();
                let running = self.running_tasks.clone();
                let agent_mgr = self.agent_manager.clone();
                let notify = self.notify.clone();
                let metrics = self.metrics.clone();

                // □□□□
                tokio::spawn(async move {
                    loop {
                        notify.notified().await;

                        while let Some(queued) = queue.lock().await.pop() {
                            match Self::try_schedule(&queued,
&agent_mgr).await {
                                Ok(handle) => {

running.write().await.insert(queued.task.id, handle);
                                    metrics.tasks_scheduled.inc();
                                }
                                Err(e) => {
                                    // □□□□□□□□□□
                                    if queued.retries < config.max_retries {
                                        let mut retry = queued;
                                        retry.retries += 1;
                                        queue.lock().await.push(retry);
                                    } else {
                                        metrics.tasks_failed.inc();
                                    }
                                }
                            }
                        }
                    }
                });

                // □□□□□□
                let running_check = self.running_tasks.clone();
                let timeout_interval = self.config.timeout_check_interval;
                tokio::spawn(async move {
                    let mut ticker = interval(timeout_interval);
                    loop {
                        ticker.tick().await;
                        Self::check_timeouts(&running_check).await;
                    }
                });
            }

            /// □□□□
            pub async fn submit(&self, task: Task) -> Result<TaskId,
SchedulerError> {
                // □□□□□□
                let queue_len = self.task_queue.lock().await.len();
                if queue_len >= self.config.queue_capacity {
                    return Err(SchedulerError::QueueFull);
                }

                let priority = match task.priority {
```

```
            TaskPriority::Critical => 0,
            TaskPriority::High => 1,
            TaskPriority::Normal => 2,
            TaskPriority::Low => 3,
        };

        let queued = QueuedTask {
            priority,
            enqueued_at: Instant::now(),
            task: task.clone(),
            retries: 0,
        };

        self.task_queue.lock().await.push(queued);
        self.metrics.tasks_submitted.inc();

        // □□□□□
        self.notify.notify_one();

        Ok(task.id)
    }

    /// □□□□□□
    async fn try_schedule(
        queued: &QueuedTask,
        agent_mgr: &Arc<dyn AgentManager>,
    ) -> Result<TaskHandle, SchedulerError> {
        let task = &queued.task;

        // 1. □□□□□□□ Agent
        let candidates = agent_mgr
            .find_candidates(&task.required_capabilities)
            .await?;

        if candidates.is_empty() {
            // □□□□ Agent□□□□□□□
            let new_agent = agent_mgr.create_for_task(task).await?;
            return Self::assign_task(task, new_agent).await;
        }

        // 2. □□□□□□□□□□ Agent
        let selected = Self::select_agent(&candidates, task).await?;

        // 3. □□□□
        Self::assign_task(task, selected).await
    }

    /// □□□□ Agent
    async fn select_agent(
        candidates: &[AgentInstance],
        task: &Task,
    ) -> Result<AgentInstance, SchedulerError> {
        // □□□□□□□ + □□□
        let mut scored: Vec<_> = candidates
            .iter()
            .map(|agent| {
                let load_score = agent.resource_usage.cpu_millicores
as f64
                    / agent.resource_quota.cpu_limit as f64;
```

```
                        // □□□□□□□□□□□□□□
                        let affinity_score = if
task.labels.get("session_id").map(|s| {
                            agent.sessions.iter().any(|sid| sid.to_string()
== *s)
                        }).unwrap_or(false) {
                            0.2
                        } else {
                            0.0
                        };

                        let score = load_score - affinity_score;
                        (agent.clone(), score)
                    })
                    .collect();

                scored.sort_by(|a, b| a.1.partial_cmp(&b.1).unwrap());

                scored
                    .into_iter()
                    .next()
                    .map(|(agent, _)| agent)
                    .ok_or(SchedulerError::NoAvailableAgent)
            }

            /// □□□□□ Agent
            async fn assign_task(
                task: &Task,
                agent: AgentInstance,
            ) -> Result<TaskHandle, SchedulerError> {
                // □□□□□ Agent
                // □□□□□□□□□□□□□□□□□
                let handle = TaskHandle {
                    task_id: task.id,
                    agent_id: agent.id,
                    started_at: Instant::now(),
                    timeout: task.timeout,
                };

                Ok(handle)
            }

            /// □□□□□□□
            async fn check_timeouts(running: &Arc<RwLock<HashMap<TaskId,
TaskHandle>>>) {
                let now = Instant::now();
                let mut to_cancel = Vec::new();

                {
                    let running_lock = running.read().await;
                    for (task_id, handle) in running_lock.iter() {
                        if now.duration_since(handle.started_at) >
handle.timeout {
                            to_cancel.push(*task_id);
                        }
                    }
                }

                for task_id in to_cancel {
                    // □□□□□□□
```

```
                // ...
            }
        }
    }

    /// □□□□
    #[derive(Debug)]
    struct TaskHandle {
        task_id: TaskId,
        agent_id: AgentId,
        started_at: Instant,
        timeout: Duration,
    }

    /// Agent □□ trait
    #[async_trait::async_trait]
    pub trait AgentManager: Send + Sync {
        /// □□□□□□□□□□ Agent
        async fn find_candidates(
            &self,
            capabilities: &[String],
        ) -> Result<Vec<AgentInstance>, SchedulerError>;

        /// □□□□□□□□ Agent
        async fn create_for_task(&self, task: &Task) ->
Result<AgentInstance, SchedulerError>;
    }

    /// □□□□□
    #[derive(Debug, Default)]
    struct SchedulerMetrics {
        tasks_submitted: Counter,
        tasks_scheduled: Counter,
        tasks_completed: Counter,
        tasks_failed: Counter,
        tasks_timeout: Counter,
        queue_depth: Gauge,
        scheduling_latency: Histogram,
    }
```

3.4 pswarm-network - QUIC □□□

: crates/pswarm-network/src/lib.rs

```
    //! QUIC □□□□□
    //!
    //! □□ quinn □□□□□□□□□□ Agent □□□

    use std::collections::HashMap;
    use std::net::SocketAddr;
    use std::sync::Arc;
    use tokio::sync::{mpsc, RwLock};
    use quinn::{ClientConfig, Connection, Endpoint, ServerConfig};
    use rustls::pki_types::{CertificateDer, PrivateKeyDer};

    use pswarm_core::*;

    /// □□□□
    #[derive(Debug, Clone)]
```

```rust
pub struct NetworkConfig {
    /// □□□□
    pub bind_addr: SocketAddr,
    /// □□□□
    pub cert_path: std::path::PathBuf,
    /// □□□□
    pub key_path: std::path::PathBuf,
    /// □□□□□□□
    pub max_connections: usize,
    /// □□□□
    pub connection_timeout: Duration,
    /// □□□
    pub stream_timeout: Duration,
    /// □□□□
    pub heartbeat_interval: Duration,
}

/// QUIC □□□□□□
pub struct QuicTransport {
    /// □□□□
    endpoint: Endpoint,
    /// □□□□
    connections: Arc<RwLock<HashMap<AgentId, Connection>>>,
    /// □□□□□
    message_rx: mpsc::Receiver<(AgentId, Message)>,
    /// □□
    config: NetworkConfig,
}

impl QuicTransport {
    pub async fn new(config: NetworkConfig) -> Result<Self,
NetworkError> {
        // □□□□
        let (cert, key) = Self::load_certs(&config.cert_path,
&config.key_path).await?;

        // □□ TLS
        let tls_config = Self::configure_tls(cert, key)?;

        // □□□□
        let endpoint = Endpoint::server(config.bind_addr,
tls_config)?;

        let (tx, rx) = mpsc::channel(1000);
        let transport = Self {
            endpoint,
            connections: Arc::new(RwLock::new(HashMap::new())),
            message_rx: rx,
            config,
        };

        // □□□□□□□□
        transport.start_accept_loop(tx).await;

        Ok(transport)
    }

    /// □□□□□ Agent
    pub async fn connect(&self, agent_id: AgentId, addr: SocketAddr)
-> Result<(), NetworkError> {
```

```
                let connection = self.endpoint.connect(addr,
"swarm.local")?.await?;

                self.connections.write().await.insert(agent_id, connection);

                // □□□□□□□□□
                let conn = self
                    .connections
                    .read()
                    .await
                    .get(&agent_id)
                    .cloned()
                    .ok_or(NetworkError::Unreachable(agent_id))?;

                self.start_receive_loop(agent_id, conn).await;

                Ok(())
            }

            /// □□□□□□□□ Agent
            pub async fn send_to(&self, agent_id: AgentId, message: Message)
-> Result<(), NetworkError> {
                let conn = self
                    .connections
                    .read()
                    .await
                    .get(&agent_id)
                    .cloned()
                    .ok_or(NetworkError::Unreachable(agent_id))?;

                // □□□□□
                let (mut send, _recv) = conn.open_bi().await?;

                // □□□□□
                let data = serde_json::to_vec(&message)?;

                // □□
                send.write_all(&data).await?;
                send.finish()?;

                Ok(())
            }

            /// □□□□
            pub async fn broadcast(&self, message: Message) -> Result<(),
NetworkError> {
                let connections = self.connections.read().await;

                for (agent_id, _) in connections.iter() {
                    // □□□□□□□□□□□□□□□□
                    let msg = message.clone();
                    let agent_id = *agent_id;

                    // □□□□
                    let connections = self.connections.clone();
                    tokio::spawn(async move {
                        if let Some(conn) =
connections.read().await.get(&agent_id) {
                            // ... □□□□
                        }
```

```
                    });
                }

                Ok(())
            }

            /// □□□□□□□□□
            pub async fn receive(&mut self) -> Result<(AgentId, Message),
NetworkError> {
                self.message_rx
                    .recv()
                    .await
                    .ok_or(NetworkError::Other(anyhow::anyhow!("Channel
closed")))
            }

            /// □□□□□□□□□
            async fn start_accept_loop(&self, tx: mpsc::Sender<(AgentId,
Message)>) {
                while let Some(conn) = self.endpoint.accept().await {
                    let tx = tx.clone();
                    tokio::spawn(async move {
                        match conn.await {
                            Ok(connection) => {
                                // □□□□
                                Self::handle_connection(connection,
tx).await;
                            }
                            Err(e) => {
                                tracing::error!("Connection failed: {}", e);
                            }
                        }
                    });
                }
            }

            /// □□□□□□
            async fn handle_connection(
                connection: Connection,
                tx: mpsc::Sender<(AgentId, Message)>,
            ) {
                loop {
                    match connection.accept_bi().await {
                        Ok((send, recv)) => {
                            // □□□□□
                            tokio::spawn(async move {
                                // □□□□□□□□□□□□□
                                // ...
                            });
                        }
                        Err(_) => break,
                    }
                }
            }

            /// □□□□□□□□□
            async fn start_receive_loop(&self, agent_id: AgentId,
connection: Connection) {
                // □□□□□□
                // ...
```

```
            }

            /// □□□□□
            async fn load_certs(
                cert_path: &std::path::Path,
                key_path: &std::path::Path,
            ) -> Result<(Vec<CertificateDer<'static>>,
PrivateKeyDer<'static>), NetworkError> {
                // □□□□□□□
                todo!()
            }

            /// □□ TLS
            fn configure_tls(
                cert: Vec<CertificateDer<'static>>,
                key: PrivateKeyDer<'static>,
            ) -> Result<ServerConfig, NetworkError> {
                // □□ TLS □□
                todo!()
            }
        }

        /// Gossip □□□□□□□□□□ + □□□□□
        pub struct GossipProtocol {
            /// □□□□
            nodes: Arc<RwLock<HashMap<NodeId, NodeInfo>>>,
            /// □□□□□□□□□□□□□□□
            seen_messages: Arc<RwLock<lru::LruCache<Uuid, ()>>>,
            /// □□□
            transport: Arc<dyn NetworkTransport>,
        }

        impl GossipProtocol {
            /// □□□□
            pub async fn gossip(&self, message: GossipMessage) -> Result<(),
NetworkError> {
                // □□□□□□□□
                if self.seen_messages.write().await.put(message.id,
()).is_some() {
                    return Ok(());  // □□□□□□□
                }

                // □□□□□□□□□□□□□□ 3-5 □□
                let targets = self.select_gossip_targets(3).await;

                // □□□□□□□□
                for node_id in targets {
                    // ...
                }

                Ok(())
            }

            /// □□ Gossip □□□□□□□□□□□□□□□□□
            async fn select_gossip_targets(&self, count: usize) ->
Vec<NodeId> {
                let nodes = self.nodes.read().await;
                let mut candidates: Vec<_> =
nodes.keys().copied().collect();
```

```rust
        // □□□□
        use rand::seq::SliceRandom;
        candidates.shuffle(&mut rand::thread_rng());

        candidates.into_iter().take(count).collect()
    }
}

/// Gossip □□□□
#[derive(Debug, Clone, Serialize, Deserialize)]
pub enum GossipMessage {
    /// □□□□
    NodeJoin(NodeInfo),
    /// □□□□
    NodeHeartbeat { node_id: NodeId, timestamp: u64 },
    /// Agent □□□□
    AgentUpdate(AgentInstance),
    /// □□□□□□
    TaskUpdate(TaskStatus),
    /// □□□□□□□
    AppMessage { channel: String, payload: Vec<u8> },
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct NodeInfo {
    pub id: NodeId,
    pub address: SocketAddr,
    pub region: String,
    pub capacity: ResourceQuota,
    pub available: ResourceQuota,
}
```

3.5 pswarm-api - HTTP/gRPC API

: crates/pswarm-api/src/lib.rs

```rust
//! HTTP/gRPC API □
//!
//! □□□□ REST API □ gRPC □□

use axum::{
    extract::{Path, State, Json},
    http::StatusCode,
    response::IntoResponse,
    routing::{get, post, put, delete},
    Router,
};
use std::sync::Arc;
use tower_http::trace::TraceLayer;

use pswarm_core::*;

/// API □□
#[derive(Clone)]
pub struct ApiState {
    /// □□□
    scheduler: Arc<dyn TaskScheduler>,
    /// Agent □□□□□
    agent_lifecycle: Arc<dyn AgentLifecycle>,
```

```rust
        /// □□□□
        storage: Arc<dyn Storage>,
    }

    /// □□ API □□
    pub fn create_router(state: ApiState) -> Router {
        Router::new()
            // Agent □□
            .route("/api/v1/agents", post(create_agent))
            .route("/api/v1/agents", get(list_agents))
            .route("/api/v1/agents/:id", get(get_agent))
            .route("/api/v1/agents/:id", delete(delete_agent))
            .route("/api/v1/agents/:id/start", post(start_agent))
            .route("/api/v1/agents/:id/stop", post(stop_agent))
            .route("/api/v1/agents/:id/pause", post(pause_agent))
            .route("/api/v1/agents/:id/resume", post(resume_agent))
            // □□□□
            .route("/api/v1/tasks", post(submit_task))
            .route("/api/v1/tasks/:id", get(get_task))
            .route("/api/v1/tasks/:id/cancel", post(cancel_task))
            // □□□□
            .route("/api/v1/sessions", post(create_session))
            .route("/api/v1/sessions/:id", get(get_session))
            .route("/api/v1/sessions/:id/messages", post(send_message))
            // □□
            .route("/api/v1/health", get(health_check))
            .route("/api/v1/metrics", get(get_metrics))
            // □□□
            .layer(TraceLayer::new_for_http())
            .with_state(state)
    }

    // ==================== Agent API ====================

    async fn create_agent(
        State(state): State<ApiState>,
        Json(spec): Json<AgentSpec>,
    ) -> Result<impl IntoResponse, ApiError> {
        // □□□□
        spec.validate().map_err(|e|
ApiError::BadRequest(e.to_string()))?;

        // □□ Agent
        let instance = state.agent_lifecycle.create(spec).await?;

        Ok((StatusCode::CREATED, Json(instance)))
    }

    async fn list_agents(
        State(state): State<ApiState>,
        // query: Query<ListAgentsQuery>,
    ) -> Result<impl IntoResponse, ApiError> {
        let filter = AgentFilter::default();  // □ query □□
        let agents = state.agent_lifecycle.list_agents(filter).await?;

        Ok(Json(agents))
    }

    async fn get_agent(
        State(state): State<ApiState>,
```

```
    Path(id): Path<AgentId>,
) -> Result<impl IntoResponse, ApiError> {
    let agent = state.agent_lifecycle.get_status(id).await?;

    Ok(Json(agent))
}

async fn delete_agent(
    State(state): State<ApiState>,
    Path(id): Path<AgentId>,
) -> Result<impl IntoResponse, ApiError> {
    state.agent_lifecycle.terminate(id).await?;

    Ok(StatusCode::NO_CONTENT)
}

async fn start_agent(
    State(state): State<ApiState>,
    Path(id): Path<AgentId>,
) -> Result<impl IntoResponse, ApiError> {
    state.agent_lifecycle.start(id).await?;

    Ok(StatusCode::OK)
}

async fn stop_agent(
    State(state): State<ApiState>,
    Path(id): Path<AgentId>,
) -> Result<impl IntoResponse, ApiError> {
    state.agent_lifecycle.terminate(id).await?;

    Ok(StatusCode::OK)
}

async fn pause_agent(
    State(state): State<ApiState>,
    Path(id): Path<AgentId>,
) -> Result<impl IntoResponse, ApiError> {
    state.agent_lifecycle.pause(id).await?;

    Ok(StatusCode::OK)
}

async fn resume_agent(
    State(state): State<ApiState>,
    Path(id): Path<AgentId>,
) -> Result<impl IntoResponse, ApiError> {
    state.agent_lifecycle.resume(id).await?;

    Ok(StatusCode::OK)
}

// ==================== Task API ====================

async fn submit_task(
    State(state): State<ApiState>,
    Json(task): Json<SubmitTaskRequest>,
) -> Result<impl IntoResponse, ApiError> {
    let task = Task {
        id: TaskId::new_v4(),
```

```
                parent_id: task.parent_id,
                task_type: task.task_type,
                input: task.input,
                priority: task.priority.unwrap_or(TaskPriority::Normal),
                timeout: task.timeout.unwrap_or(Duration::from_secs(300)),
                required_capabilities: task.required_capabilities,
                labels: task.labels.unwrap_or_default(),
                created_at: SystemTime::now(),
            };

            let task_id = state.scheduler.submit(task).await?;

            Ok((StatusCode::CREATED, Json(SubmitTaskResponse { task_id })))
        }

        async fn get_task(
            State(state): State<ApiState>,
            Path(id): Path<TaskId>,
        ) -> Result<impl IntoResponse, ApiError> {
            let status = state.scheduler.get_status(id).await?;

            Ok(Json(status))
        }

        async fn cancel_task(
            State(state): State<ApiState>,
            Path(id): Path<TaskId>,
        ) -> Result<impl IntoResponse, ApiError> {
            state.scheduler.cancel(id).await?;

            Ok(StatusCode::OK)
        }

        // ==================== Session API ====================

        async fn create_session(
            State(state): State<ApiState>,
            Json(req): Json<CreateSessionRequest>,
        ) -> Result<impl IntoResponse, ApiError> {
            let session = Session {
                id: SessionId::new_v4(),
                root_task_id: req.initial_task_id,
                agents: vec![req.initial_agent_id],
                current_agent: Some(req.initial_agent_id),
                messages: vec![],
                context_variables:
req.context_variables.unwrap_or_default(),
                created_at: SystemTime::now(),
                updated_at: SystemTime::now(),
            };

            state.storage.save_session(&session).await?;

            Ok((StatusCode::CREATED, Json(session)))
        }

        async fn get_session(
            State(state): State<ApiState>,
            Path(id): Path<SessionId>,
        ) -> Result<impl IntoResponse, ApiError> {
```

```rust
    let session = state.storage.get_session(id).await?;

    Ok(Json(session))
}

async fn send_message(
    State(state): State<ApiState>,
    Path(id): Path<SessionId>,
    Json(req): Json<SendMessageRequest>,
) -> Result<impl IntoResponse, ApiError> {
    // □□□□
    let mut session = state.storage.get_session(id).await?;

    // □□□□
    let message = Message {
        id: Uuid::new_v4(),
        sender: req.sender,
        recipient: req.recipient,
        message_type: req.message_type,
        content: req.content,
        timestamp: SystemTime::now(),
    };

    // □□□□□
    session.messages.push(message.clone());
    session.updated_at = SystemTime::now();

    // □□
    state.storage.save_session(&session).await?;

    // □□□□□ Agent
    if let Some(recipient) = req.recipient {
        // □□ recipient □ AgentId
        let agent_id = AgentId::parse_str(&recipient)?;
        // □□□□□□□
        // ...
    }

    Ok(Json(message))
}

// ==================== System API ====================

async fn health_check() -> impl IntoResponse {
    Json(serde_json::json!({
        "status": "healthy",
        "version": env!("CARGO_PKG_VERSION"),
        "timestamp": chrono::Utc::now().to_rfc3339(),
    }))
}

async fn get_metrics() -> impl IntoResponse {
    // □□ Prometheus □□□□□
    "# metrics here"
}

// ==================== □□/□□□□ ====================

#[derive(Debug, Deserialize)]
struct SubmitTaskRequest {
```

```rust
    parent_id: Option<TaskId>,
    task_type: String,
    input: serde_json::Value,
    priority: Option<TaskPriority>,
    timeout: Option<Duration>,
    required_capabilities: Vec<String>,
    labels: Option<HashMap<String, String>>,
}

#[derive(Debug, Serialize)]
struct SubmitTaskResponse {
    task_id: TaskId,
}

#[derive(Debug, Deserialize)]
struct CreateSessionRequest {
    initial_task_id: TaskId,
    initial_agent_id: AgentId,
    context_variables: Option<HashMap<String, serde_json::Value>>,
}

#[derive(Debug, Deserialize)]
struct SendMessageRequest {
    sender: String,
    recipient: Option<String>,
    message_type: MessageType,
    content: serde_json::Value,
}

// ==================== □□□□ ====================

#[derive(Debug, thiserror::Error)]
enum ApiError {
    #[error("Bad request: {0}")]
    BadRequest(String),

    #[error("Not found")]
    NotFound,

    #[error(transparent)]
    Agent(#[from] AgentError),

    #[error(transparent)]
    Scheduler(#[from] SchedulerError),

    #[error(transparent)]
    Storage(#[from] StorageError),

    #[error(transparent)]
    Other(#[from] anyhow::Error),
}

impl IntoResponse for ApiError {
    fn into_response(self) -> axum::response::Response {
        let (status, body) = match self {
            ApiError::BadRequest(msg) => (
                StatusCode::BAD_REQUEST,
                Json(serde_json::json!({ "error": msg })),
            ),
            ApiError::NotFound => (
```

```
                StatusCode::NOT_FOUND,
                Json(serde_json::json!({ "error": "Not found" })),
            ),
            ApiError::Agent(e) => (
                StatusCode::INTERNAL_SERVER_ERROR,
                Json(serde_json::json!({ "error": e.to_string() })),
            ),
            ApiError::Scheduler(e) => (
                StatusCode::SERVICE_UNAVAILABLE,
                Json(serde_json::json!({ "error": e.to_string() })),
            ),
            ApiError::Storage(e) => (
                StatusCode::INTERNAL_SERVER_ERROR,
                Json(serde_json::json!({ "error": e.to_string() })),
            ),
            ApiError::Other(e) => (
                StatusCode::INTERNAL_SERVER_ERROR,
                Json(serde_json::json!({ "error": e.to_string() })),
            ),
        };

        (status, body).into_response()
    }
}
```

---

4. □□□□

4.1 □□□□

- □□cargo fmt □□□□□
- □□cargo clippy □□□□
- □□□□□□//!
- □□□□□thiserror □ anyhow

4.2 □□□□

```
// □□□□
#[cfg(test)]
mod tests {
    use super::*;

    #[tokio::test]
    async fn test_scheduler_submit() {
        let scheduler = create_test_scheduler().await;
        let task = create_test_task();

        let task_id = scheduler.submit(task).await.unwrap();

        assert!(!task_id.is_nil());
    }

    #[tokio::test]
    async fn test_agent_lifecycle() {
        let manager = create_test_manager().await;
        let spec = create_test_spec();

        let agent = manager.create(spec).await.unwrap();
```

```
            assert_eq!(agent.state, AgentState::Creating);

            manager.start(agent.id).await.unwrap();
            let status = manager.get_status(agent.id).await.unwrap();
            assert_eq!(status.state, AgentState::Idle);

            manager.terminate(agent.id).await.unwrap();
        }
    }

    // □□□□
    #[tokio::test]
    async fn test_end_to_end_task_execution() {
        // □□□□□□□
        let (scheduler, agent_mgr) = start_test_cluster().await;

        // □□ Agent
        let spec = AgentSpec {
            // ...
        };
        let agent = agent_mgr.create(spec).await.unwrap();

        // □□□□
        let task = Task {
            // ...
        };
        let task_id = scheduler.submit(task).await.unwrap();

        // □□□□
        let status =
scheduler.wait_for_completion(task_id).await.unwrap();

        assert_eq!(status.state, TaskState::Completed);
    }
```

4.3 □□□□

```
    use tracing::{info, warn, error, debug, span, Level};

    // □□□□□
    info!(
        agent_id = %agent_id,
        task_id = %task_id,
        duration_ms = execution_time,
        "Task completed successfully"
    );

    // Span □□
    let span = span!(Level::INFO, "agent_execution", agent_id =
%agent_id);
    let _enter = span.enter();
```

---

5. □□□□

5.1 □□□□

```
    # docker-compose.yml
```

```yaml
version: '3.8'
services:
  pswarm-api:
    image: paperclip/pswarm:latest
    command: ["api-server"]
    ports:
      - "8080:8080"
    environment:
      - RUST_LOG=info
      - DATABASE_URL=postgres://pswarm:secret@postgres:5432/pswarm
      - REDIS_URL=redis://redis:6379
    depends_on:
      - postgres
      - redis

  pswarm-worker:
    image: paperclip/pswarm:latest
    command: ["worker"]
    environment:
      - RUST_LOG=info
      - API_ENDPOINT=http://pswarm-api:8080
    deploy:
      replicas: 2

  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: pswarm
      POSTGRES_PASSWORD: secret
      POSTGRES_DB: pswarm
    volumes:
      - postgres_data:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine
    volumes:
      - redis_data:/data

volumes:
  postgres_data:
  redis_data:
```

5.2 Kubernetes □□

```yaml
# deployments/k8s/operator.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: pswarm-operator
  namespace: pswarm-system
spec:
  replicas: 1
  selector:
    matchLabels:
      app: pswarm-operator
  template:
    metadata:
      labels:
        app: pswarm-operator
    spec:
```

```yaml
      serviceAccountName: pswarm-operator
      containers:
        - name: operator
          image: paperclip/pswarm-operator:latest
          resources:
            limits:
              cpu: 500m
              memory: 512Mi
          env:
            - name: WATCH_NAMESPACE
              value: ""
---
# CRD □□
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: agents.pswarm.io
spec:
  group: pswarm.io
  versions:
    - name: v1
      served: true
      storage: true
      schema:
        openAPIV3Schema:
          type: object
          properties:
            spec:
              type: object
              properties:
                runtime:
                  type: object
                  properties:
                    type:
                      type: string
                      enum: [wasm, container]
                    image:
                      type: string
                resources:
                  type: object
                  properties:
                    cpu:
                      type: string
                    memory:
                      type: string
  scope: Namespaced
  names:
    plural: agents
    singular: agent
    kind: Agent
    shortNames:
      - ag
```

---

6. □□□□□□

Phase 1: □□□□□2□□

	TBD	1□ -
pswarm-core □□□□	TBD	2□ -
pswarm-runtime WASM □□	TBD	3□ pswarm-core
pswarm-network QUIC □□	TBD	3□ pswarm-core
	TBD	1□

Phase 2: □□□□□2□□

pswarm-scheduler □□	TBD	4□ pswarm-core
pswarm-discovery □□□□	TBD	3□ pswarm-network
Agent □□□□□□	TBD	3□ pswarm-runtime
Handoff □□□□	TBD	2□ pswarm-scheduler

Phase 3: API □□□□2□□

pswarm-storage □□	TBD	3□ pswarm-core
pswarm-api REST □□	TBD	3□ pswarm-scheduler
gRPC □□	TBD	2□ pswarm-api
CLI □□	TBD	2□ pswarm-api

Phase 4: □□□□□□□□2□□

OpenTelemetry □□	TBD	2□
Prometheus □□	TBD	2□
Web UI Console	TBD	4□ pswarm-api
K8s Operator	TBD	3□ pswarm-api
Helm Chart	TBD	1□ K8s Operator

---

7. □□

7.1 □□□

Agent	
Handoff	Agent □□□□□□□□
Session	

WASM	WebAssembly□□□□□□□
QUIC	□□ UDP □□□□□□□
Gossip	

7.2 □□□□

- OpenAI Swarm
- QUIC RFC 9000
- Wasmtime Docs
- Tokio Tutorial

□□□□□□□□□□□□□□□□□□□□□□□□